

SIMATS ENGINEERING
ASSESSMENT TOOL 1- High (Saveetha Compiler Assessment
Level 3 & Viva voce)

COURSE CODE: CSA09

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO5: *Construct database-driven web applications by implementing JDBC connectivity to perform SQL operations such as insert, update, and delete using different statement types. (BL3,BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 10

Total Marks: 100

ANNEXURE A

Saveetha Compiler Assessment Level 3 & Viva voce

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANNEXURE

Saveetha Compiler Assessment Level 3 & Viva voce

1. Student Record Deletion System

Develop a Java application using JDBC to delete student records from a database based on student ID. Use PreparedStatement to ensure secure execution. Handle cases where the record does not exist. Implement proper exception handling. Test with valid and invalid IDs. Analyze the impact of delete operations on database integrity and performance.

2. Product Inventory Insert and Display System

Design a JDBC-based application to insert product details into a database and retrieve all records. Use PreparedStatement for insertion and Statement for retrieval. Validate product data before insertion. Handle SQL exceptions and display results using ResultSet. Analyze query performance and suggest optimizations.

3. Customer Record Search System

Implement a Java program using JDBC to search customer details based on customer ID or name. Use PreparedStatement to prevent SQL injection. Handle cases where no matching records are found. Validate user input. Analyze efficiency of search queries and indexing.

4. Bank Transaction Update System

Develop a JDBC application to update account balances after deposit or withdrawal operations. Use transaction management to ensure consistency. Implement rollback in case of failure. Validate input values. Analyze how transactions maintain data integrity.

5. Course Registration System

Design a Java JDBC program to insert and manage course registrations for students. Use batch processing for multiple insert operations. Handle duplicate entries and constraint violations. Test with bulk data. Analyze performance improvements using batch updates.

6. Employee Record Retrieval with Sorting

Create a JDBC application to retrieve employee records sorted by salary or name. Use SQL queries with ORDER BY clause. Handle large datasets efficiently. Analyze performance differences with and without indexing. Validate database connectivity and query execution.

7. User Authentication System

Develop a JDBC-based login system that verifies user credentials stored in a database. Use PreparedStatement for secure queries. Handle invalid login attempts and display appropriate messages. Analyze security concerns such as SQL injection and password handling.

8. Order Management System

Design a JDBC application to insert, update, and delete order records. Use different statement types (Statement, PreparedStatement). Implement proper exception handling and transaction control. Test multiple operations. Analyze system performance under concurrent access.

9. Library Database Management System

Develop a Java program using JDBC to manage book records (insert, update, delete). Validate inputs and handle duplicate entries. Use transactions for multiple operations. Analyze performance and scalability of the system.

10. Student Marks Analysis System

Create a JDBC application to insert and retrieve student marks and calculate average using SQL queries. Use PreparedStatement for secure insertion. Handle invalid inputs. Analyze query efficiency and suggest improvements.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Logic Development	25%	Develops correct, efficient, and well-structured logic for the problem	Logic is mostly correct with minor gaps	Logic is partially correct; needs improvement	Incorrect or no logical approach
Syntax & Semantics	20%	Writes error-free code with correct syntax and semantics	Minor syntax/semantic errors that do not affect execution significantly	Multiple errors; requires corrections	Frequent errors; code does not compile/execute
Time & Space Complexity	20%	Demonstrates optimal time and space complexity with proper justification	Reasonably efficient with minor scope for improvement	Limited consideration of complexity	No consideration of time or space complexity
Test Case Validation	20%	Designs and validates comprehensive test cases including edge cases	Tests cover most scenarios with minor gaps	Limited test cases; misses important cases	No or incorrect test case validation
Viva Voce	15%	Explains logic, code, and concepts clearly with confidence and depth	Explains concepts with minor gaps	Limited understanding; needs prompting	Unable to explain concepts